

3.2 Addition and Subtraction

2's Complement Operations

→ invert all bits and add 1

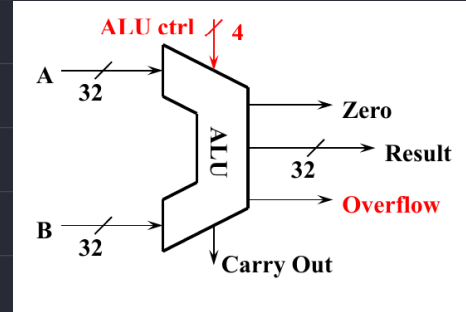
Functional Specification of ALU

ALUctrl	Function
0000	and
0001	Or
0010	add
0110	subtract
0111	set-less-than
1100	nor

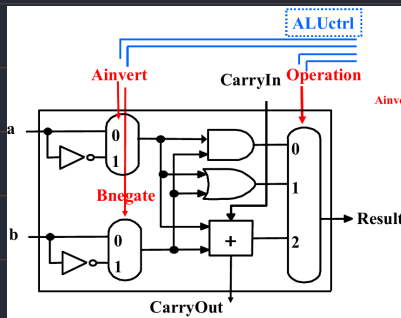
MUX Selector

Ainvert Bnegate Operation (MUX_{Selector})

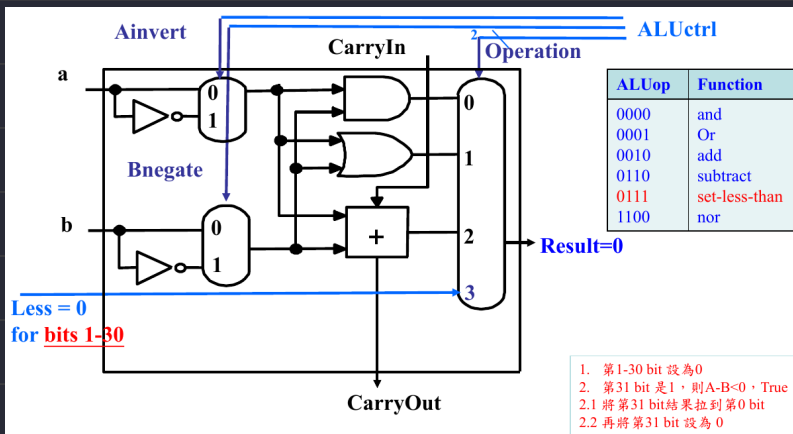
可以實作出 >、≥、<、≤



Nor Operation



Set on Less Than



$$A < B \cong A - B < 0$$

如果 $A - B < 0$ 成立 (=1)

✓ 則 $A - B$ 是負數

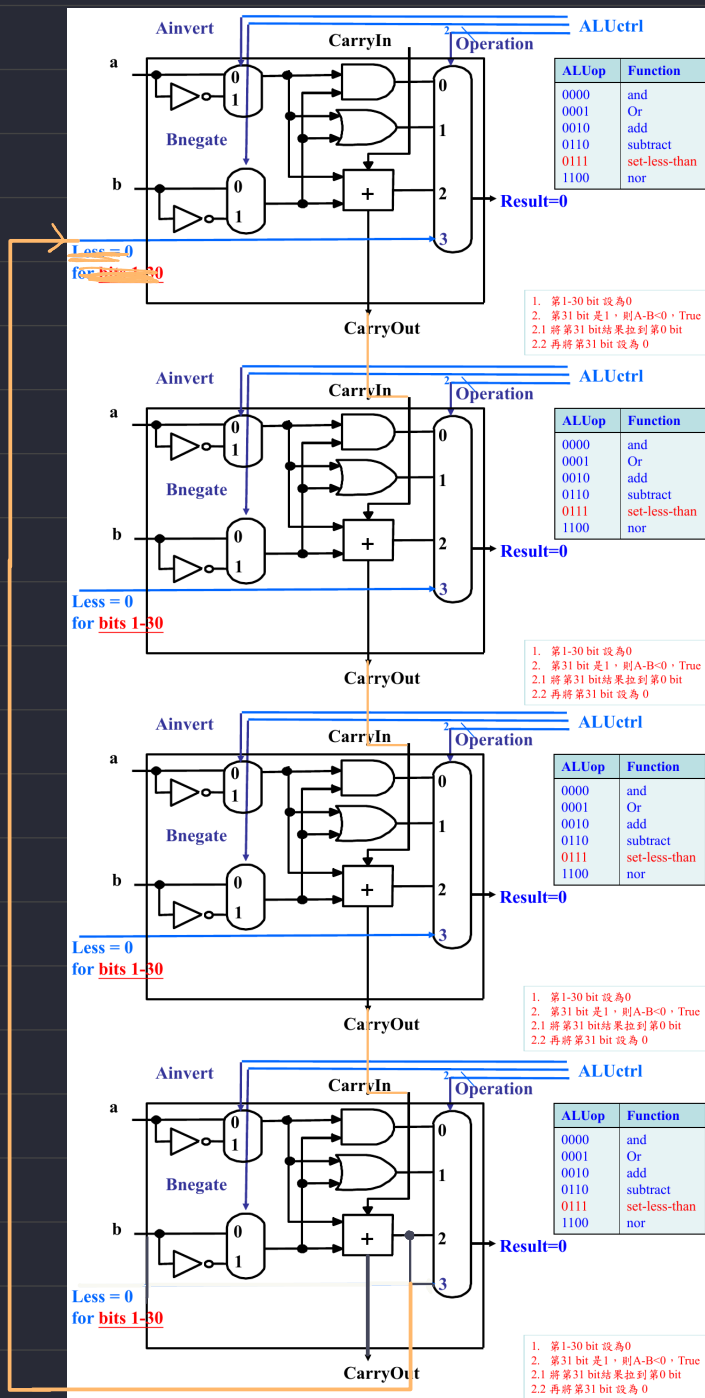
將 bit 0~30 SET '0'

bit 31 SET MSB

(2's sign)

1. 畫出 4bit ALU 的 less than 線路圖

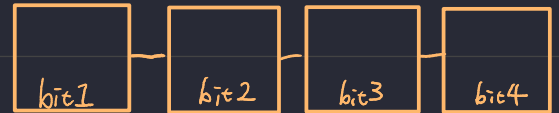
4bit ALU 的 less than 線路圖



Adder (加法器)

An ALU with Ripple Carry Adder

等待每個位元的 carry 需要額外的時間，太慢



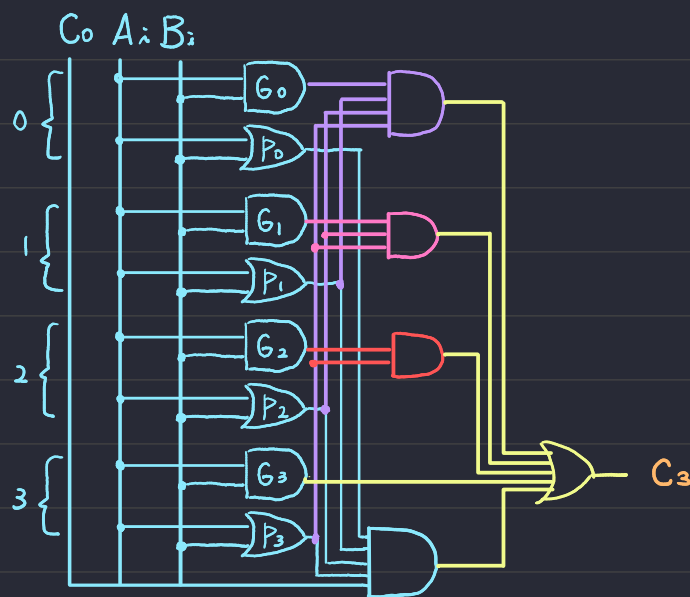
Carry Lookahead: Theory

2. 畫出第 3 bit carry out 邏輯線路圖和公式 (input: A_i, B_i, C_0)

$$G_i = A_i \times B_i \quad C_{i+1} = P_i \times C_i + G_i$$

$$P_i = A_i + B_i$$

$$\begin{aligned} C_3 &= P_3 \times C_3 + G_3 \\ &= P_3 \times (P_2 \times C_2 + G_2) + G_3 \\ &= P_3 P_2 C_2 + P_3 G_2 + G_3 \\ &= P_3 P_2 (P_1 C_1 + G_1) + P_3 G_2 + G_3 \\ &= P_3 P_2 P_1 C_1 + P_3 P_2 G_1 + P_3 G_2 + G_3 \\ &= P_3 P_2 P_1 (P_0 C_0 + G_0) + P_3 P_2 G_1 + P_3 G_2 + G_3 \\ &= P_3 P_2 P_1 P_0 C_0 + P_3 P_2 P_1 G_0 + P_3 P_2 G_1 + P_3 G_2 + G_3 \end{aligned}$$

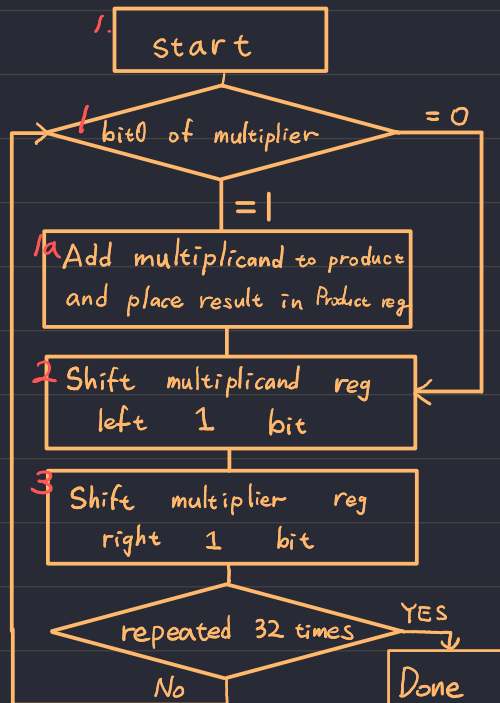


Multiplication (乘法器)

Multiplicand 被乘數 Multiplier 乘數

Version 1:

Flow Chart: (必考)



Size of register:

Name	bits
Multiplicand Register	64 bits
Multiplier Register	32 bits
ALU	64 bits
Product Register	64 bits

32bit x 32bit

Hardware implementation

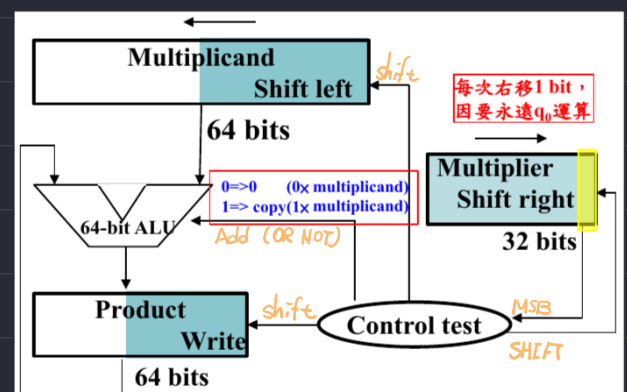


Table Chart: 必考

Ex: 0010 x 0011

Product	Multiplier	Multiplicand
0000 0000	0011	0000 0010 (S, 1)
0000 0010	0001	0000 0100 (1a, 2, 3, 1)
0000 0110	0000	0000 1000 (1a, 2, 3, 1)
0000 0110	0000	0001 0000 (2, 3, 1)
0000 0110	0000	0010 0000 (2, 3, D)

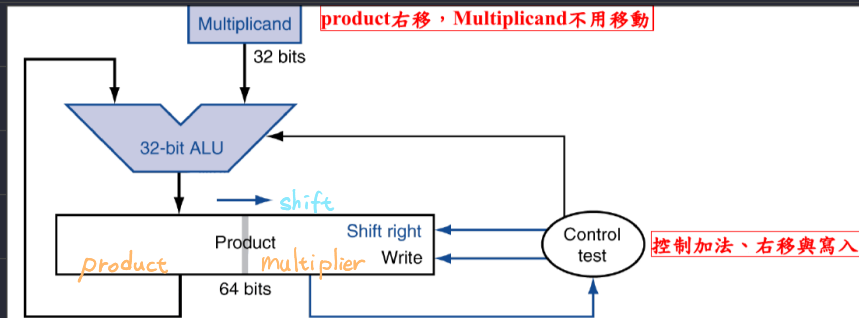
缺陷

→ Multiplicand 會一直插入0, 最後有 32 bit 用還是 0

→ Product 一開始是 0, 且 product 後面 bit 計算出後就不會改變

>> 整合起來: 讓 product 右移取代被乘數左移

Improvement of a multiplier (Version 2)



Flow Chart

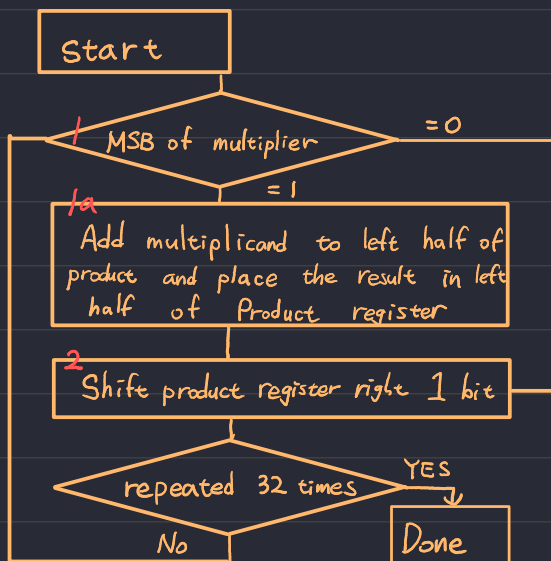


Table Chart

Example: 0010×0011

Multiplicand	Product
0010	0000 0011 (S, 1)
	0010 0011 (1a)
0010	0001 0001 (2, 1)
	0011 0001 (1a)
0010	0001 1000 (2, 1)
0010	0000 1100 (2, 1)
0010	0000 0110 (2, D)

Signed Multiplication

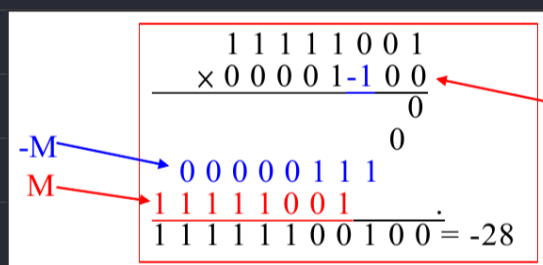
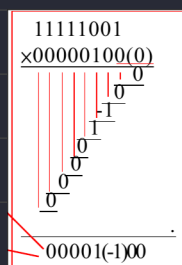
- 轉成正數後作乘法，最後再補正負號
- Booth's Algorithm (必考, version 3)

Step 1 > 將乘數後面補 0

Step 2 > 從最右邊開始 2bit 為單位向右，查表替換乘數

Step 3 > 根據查表值，把乘數(M) 與乘數的負數 (-M)

Example: $-7 (11111001) * 4 (00000100)$, $M=11111001$, $-M=00000111$



Source	Result
00	0
01	1
10	-1
11	0

陣列乘法器

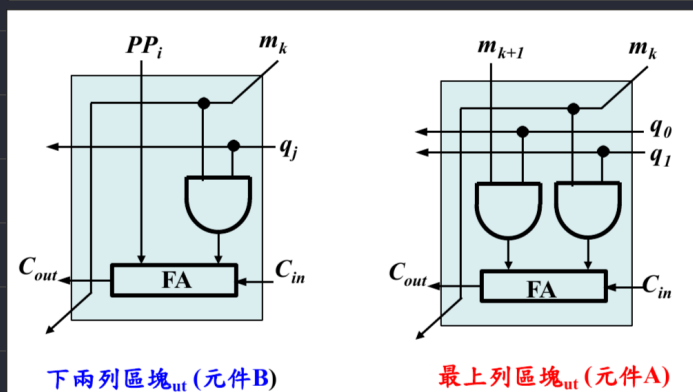
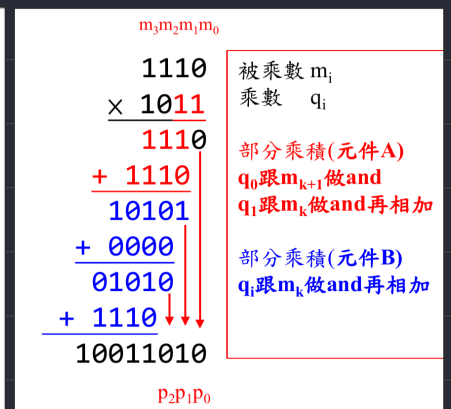
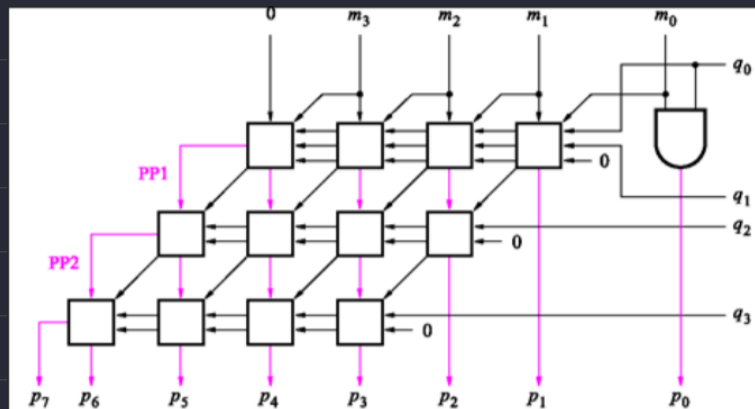
$$\text{邏輯公式: } p_i = \sum_{n=0}^i m_n \cdot q_{(i-n)}$$

$$p_0 = m_0 \cdot q_0$$

$$p_1 = m_0 \cdot q_1 \oplus m_1 \cdot q_0$$

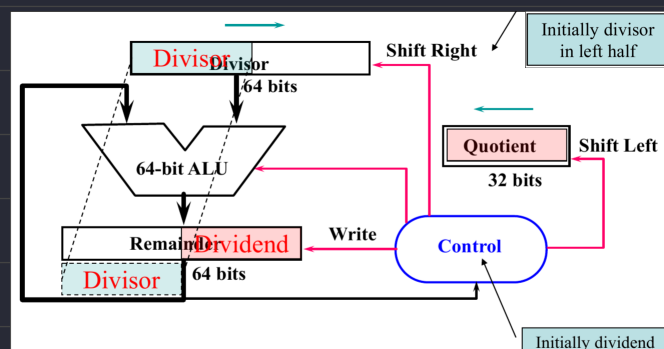
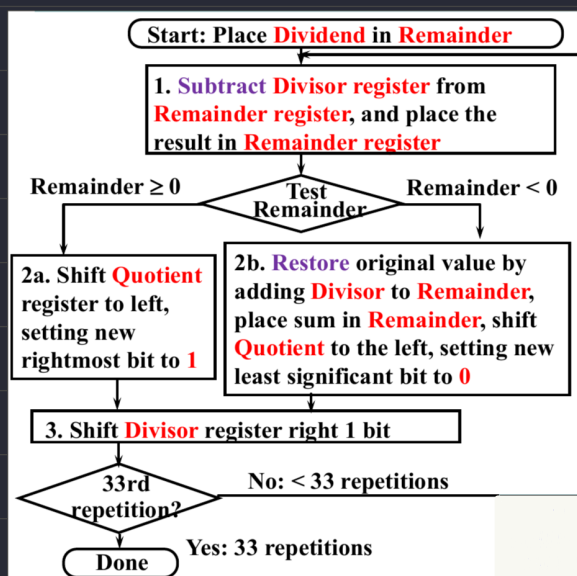
$$p_2 = m_0 \cdot q_2 \oplus m_1 \cdot q_1 \oplus m_2 \cdot q_0 \oplus C_1$$

$$p_3 = m_0 \cdot q_3 \oplus m_1 \cdot q_2 \oplus m_2 \cdot q_1 \oplus m_3 \cdot q_0 \oplus C_2$$



3.4 Division

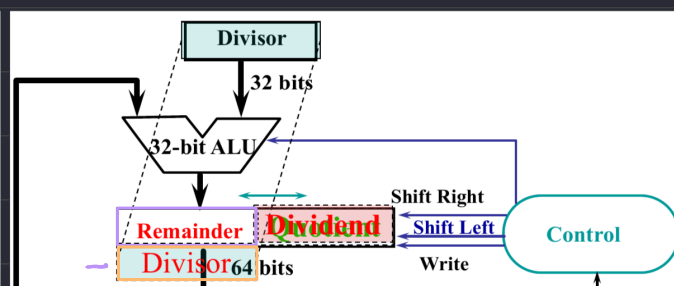
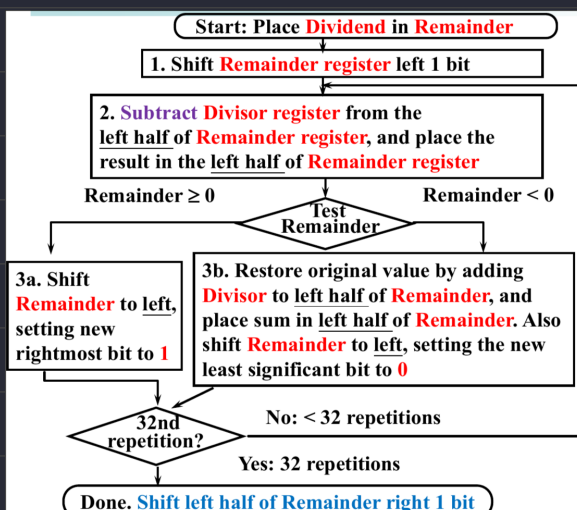
Divide Algorithm Version 1



註：33次運算是因為低一次是無效的運算，Divisor是
高32bit而Remainder是低32bit，相減結果一定
小於零，而導致 Quotient 前面被固定填入一個 0

結論：有一半的 Divisor 都是 0，可以把至些位置讓給 Quotient

Divide Hardware Version 2



註：因為步驟1的左移避免了無效的減法，所以只要做
32次，但是也因此會把 Remainder 推1+32次
造成 Quotient 正確但是 Remainder 多左移 x1

Signed Divide

轉成正數後在上除法器，最後結果再去判定正負數

Faster Division

除法器涉及每一次減法後判斷是否需要還原，無法相乘法器那樣直接加速

MIPS Division

HI: 32-bit remainder / LO: 32-bit quotient → 類似於第二版除法器

MIPS Multiply/Divide Summary

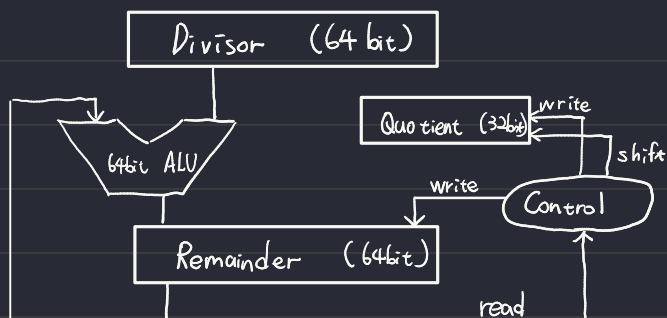
mult /div 會把結果放到 lo、hi Register，可以藉由 mflo/mfhi 取值，mtlo/mthi 寫值

Zuvio week1001 列出 0010 / 0101 的流程圖：

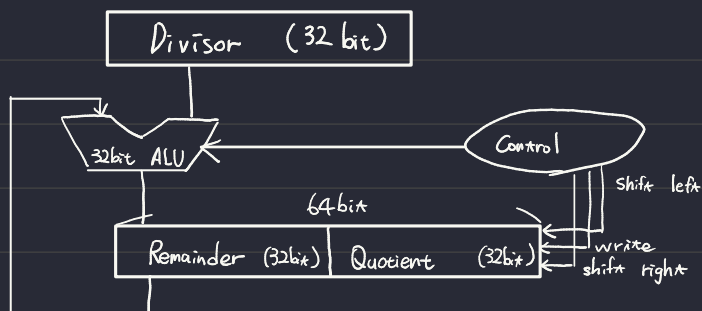
	Quot.	Divisor	Remainder
Start	0000	0010 0000	0000 0101
1.			11 10 0101
2b	0000	0010 0000	0000 0101
3	0000	0001 0000	0000 0101
1.	0000	0001 0000	1111 0101
2b	0000	0001 0000	0000 0101
3.	0000	0000 1000	0000 0101
1.	0000	0000 1000	1111 1101
2b	0000	0000 1000	0000 0101
3.	0000	0000 0100	0000 0101
1.	0000	0000 0100	0000 0001
2a	0001	0000 0100	0000 0001
3	0001	0000 0010	0000 0001
1	0001	0000 0010	1111 1111
2b	0001	0000 0010	0000 0001
3	0010	0000 0010	0000 0001
Done	0010	0000 0010	0000 0001

Zuvio week1002 劃出第一、二版除法器線路圖

Version 1



Version 2



Zuvio 1003 畫出 0111 / 0010 的流程步驟圖

Iter	Step	Remainder		Divisor
0.	Init	0000	0111	0010
1.	1.	0000	1110	0010
	2.	1110	1110	0010
	3b	0001	1100	0010
2.	2	1111	1110	0010
	3b	0011	1000	0010
3.	2.	0001	1000	0010
	3a	0011	0001	0010
4.	2.	0001	0001	0010
	3a	0010	0011	0010
Done		0001	0011	0010
		<u>Remainder</u>		<u>Quotient</u>

3.5 Floating Point

Floating Point Standard : IEEE Std 754-1985

Single float point (32bit)

S (1bit)	Exponent (8bit)	Fration(Mantissa) (23bit)	Bias = 127
----------	-----------------	---------------------------	------------

Double float point (64bit)

S (1bit)	Exponent (11bit)	Fration(Mantissa) (52bit)	Bias = 1023
----------	------------------	---------------------------	-------------

Normalized Numbers

$$x = (-1)^S \times (1 + Fraction) \times 2^{(Exponent - Bias)}$$

Denormalized Number

$$x = (-1)^S \times (0 + Fraction) \times 2^{1 - Bias}$$

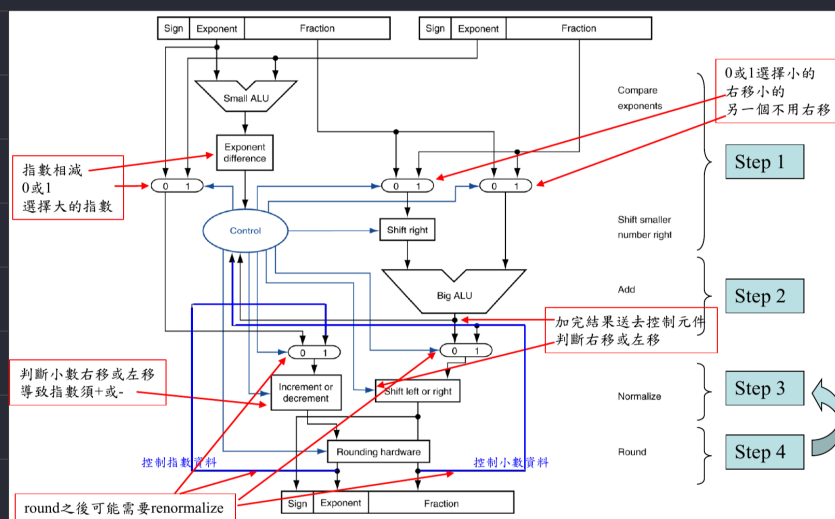
Single precision		double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	± 0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1-254	Anything	1-2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

Floating-Point Addition

Steps

1. Align binary points (比較小的要右移)
2. Add significands (小數部分相加)
3. Normalize result & check for over/underflow
4. Round and renormalize if necessary

Hardware



Floating-Point Multiplication

Steps

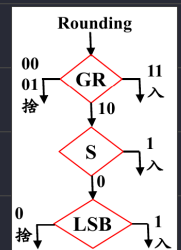
1. Add exponents
2. Multiply significands
3. Normalize result & check for over/underflow
4. Round and renormalize if necessary
5. Determine sign of result from signs of operands

Accurate Arithmetic (Rounding Numbers)

Policy: unbiased to the nearest even (4捨6入5往有效位之偶數入)

1.0100000101011

LSBGRS (右邊全部做 OR)



考試要點

考: IEEE std $\longleftrightarrow$ binary normalize

考: 十進制轉二進制 float

Zuvio week1004

1) 11101.101101 IEEE 754 single = ?

$$1.1101101101 \times 2^{-4}$$

S(1) | Exponent (8) | Fraction (23)

$$+ \quad 4 + 127 = 131 \quad \cdot \quad \underline{1101101101}$$

0 1000 0011 0000 0000 0000 0 11 0110 1101

0 10000011 000 0000 0000 0011 0110 1101

2) 10.6578 IEEE 754 single = 最接近 ?

1. 轉 binary: 1010,10101000011001011001010

2. normalize $1.01010100011001011001010 \times 2^{-3}$

3. S(1) Exponent (8) Fraction (23)

$$+ \quad -3 + 127 = 124 \quad \dots$$

01111100

0 01111100 0101 0101 0000 1100 1011 001

```
def calculate(num: float, depth: int = 0):
    if depth >= 23:
        return
```

```
    num *= 2
```

```
    if num > 1:
        print("1", end="")
        calculate(num-1, depth+1)
```

```
    else:
        print("0", end="")
        calculate(num, depth+1)
```

```
def main():
    calculate(0.6578)
    print()
    return
```

```
if __name__ == "__main__":
    main()
```

$$\begin{array}{r}
 1111 \\
 0.6578 \\
 \times \quad 2 \\
 \hline
 1.3156 \dots 1 \\
 \times \quad 2 \\
 \hline
 0.6312 \dots 0 \\
 \times \quad 2 \\
 \hline
 1.2624 \dots 1 \\
 \times \quad 2 \\
 \hline
 0.5248 \dots 0 \\
 \times \quad 2 \\
 \hline
 1.0496 \dots 1 \\
 \times \quad 2 \\
 \hline
 0.0992 \dots 0 \\
 \times \quad 2 \\
 \hline
 0.1984 \dots 0 \\
 \times \quad 2 \\
 \hline
 0.3968 \dots 0 \\
 \times \quad 2 \\
 \hline
 0.7936 \dots 0 \\
 \times \quad 2 \\
 \hline
 1.5872 \dots 1 \\
 \vdots
 \end{array}$$

3) IEEE 754 =(?)₁₀

→ 1 01010101 110110000000000000000000

$$- 85 - 127 = -42$$

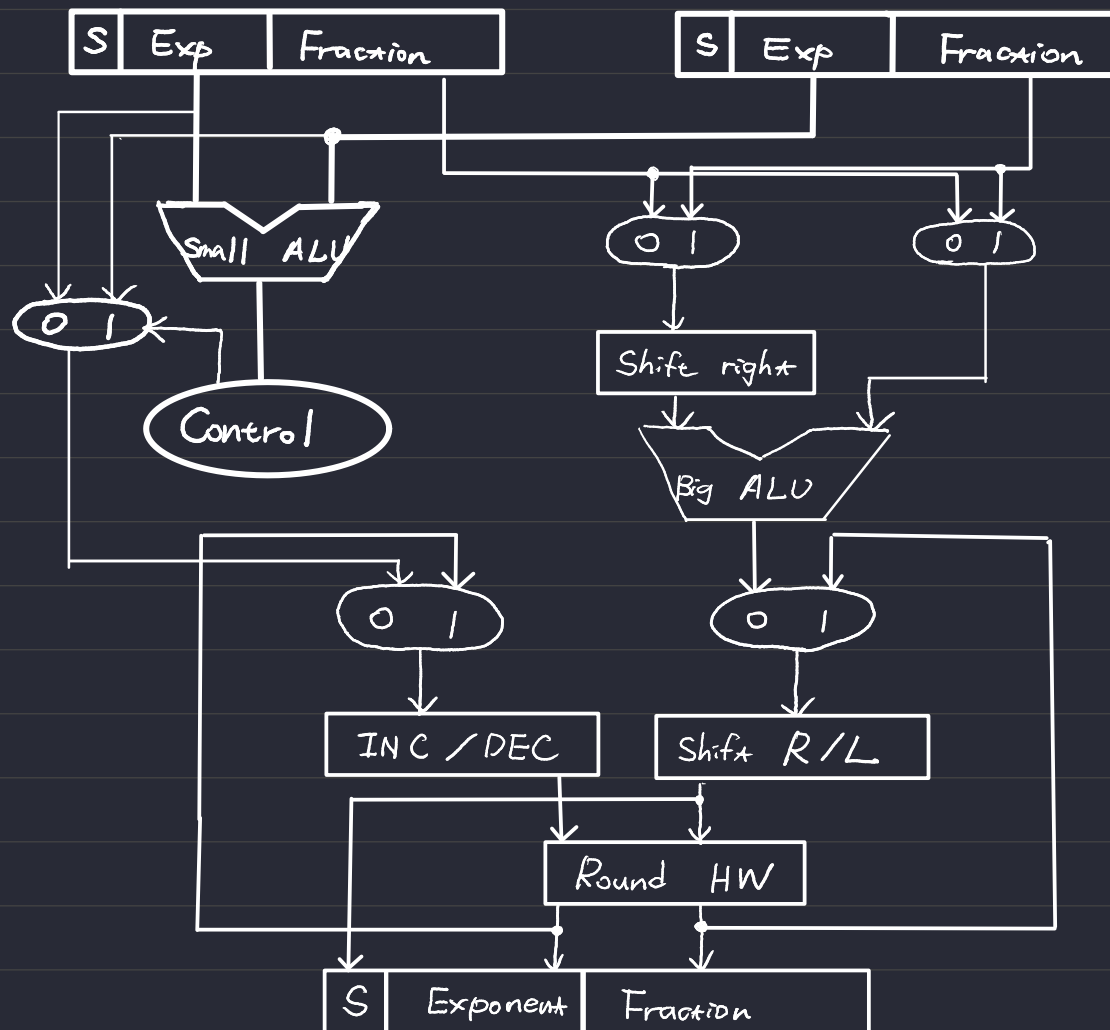
$$= - 1.11011 \times 2^{-42}$$

$$= - 2.84375 \times 2^{-42} \quad \log(2^{-42}) = -29.11$$

$$\approx - 2.84375 \times 10^{-29}$$

Zuvio week1005

畫出 FP Adder 電路圖



Zuvio 1101

(1) 1.0101101 有效位數到小數後三位

GR → 入 → 1.011

(2) 請設計 7 位小數位數，IEEE 754 的進位之五種情況的數字

五種 Cases:

1) 1.000 01 00 ~~2) 1.000 00 00~~

3) 1.000 11 00

4) 1.000 10 10

5) 1.001 10 00

6) 1.000 10 00